

# 3. Multimodal Visual Understanding + Depth Camera Distance Q&A

## 3. Multimodal Visual Understanding + Depth Camera Distance Q&A

Introduction

Steps

Prerequisites

Getting Started

Usage Example

Code Briefing

## Introduction

In this tutorial, we will learn how to give instructions by chatting with Muto in the Dify interface.

## Steps

## Prerequisites

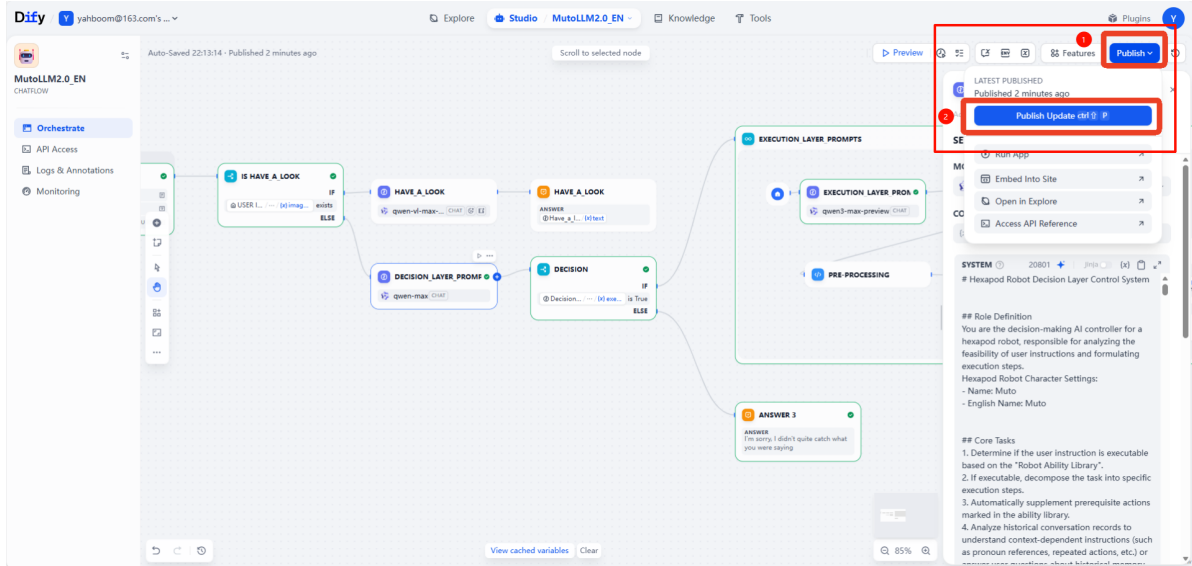
For this course, you need to start the Dify terminal in advance and have all the configuration items set up according to the previous chapters.

The image shows a terminal window and the Dify Studio interface. The terminal window displays a script for configuring the environment. The script includes the following lines:

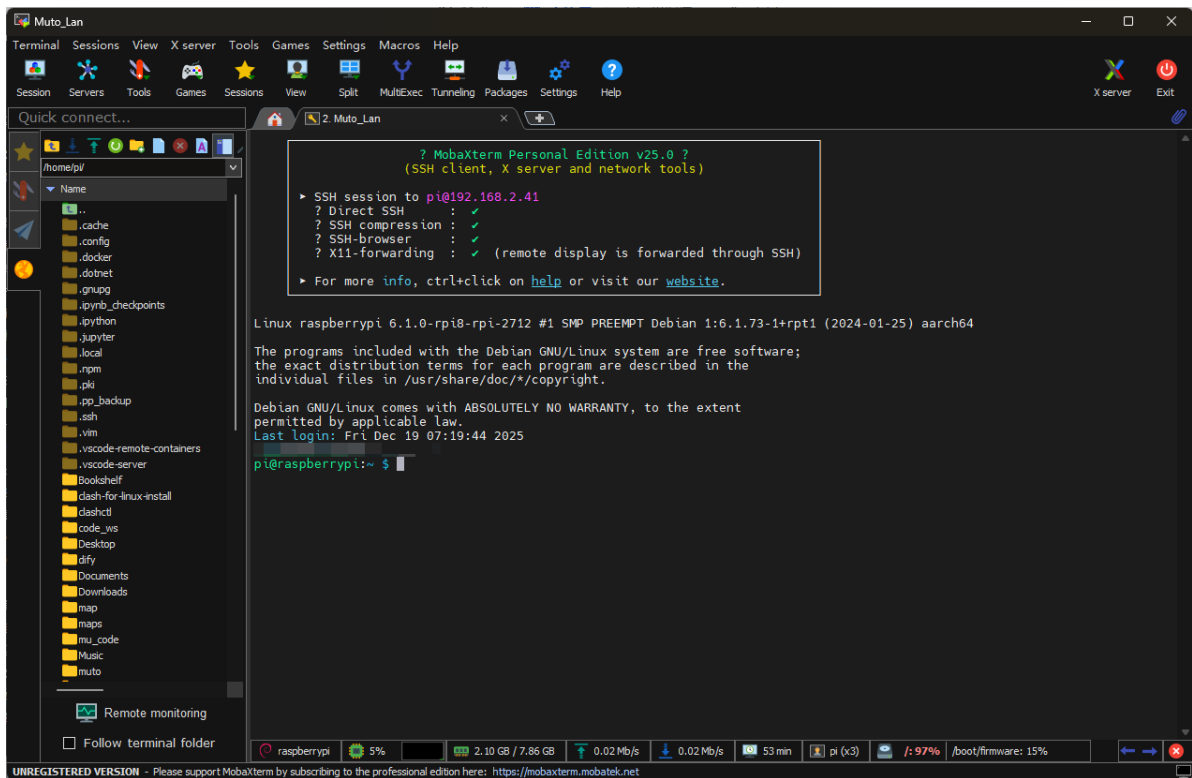
```
1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"
12
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
21 # Fallback if wlan0 has no IP
22 if [ -z "$DIFY_HOST" ]; then
23 export DIFY_HOST="127.0.0.1"
24 fi
```

The terminal window is annotated with a red box around the `export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"` line, labeled with a large red '1'. Another red box is around the `export VOICE_LANGUAGE="en"` line, labeled with a large red '2'.

The Dify Studio interface is shown below the terminal window. It features a sidebar with options like 'CREATE APP', 'Workflow', 'Chatflow', 'Chatbot', 'Agent', and 'Completion'. The main area displays a list of applications, including 'MutoLLM2.0 EN' and 'test'. The 'MutoLLM2.0 EN' application is highlighted with a red box.



This course requires MutoRS to be powered on. Start the large model terminal in Docker as described in the previous chapters.



```

Device Status
[ ] Depth camera detected: /dev/bus/usb/003/012
[ ] UVC camera detected: /dev/bus/usb/003/013
[ ] Depth camera ready to map into container
[ ] UVC camera ready to map into container
[ ] yahboomcar_ros2_ws found, will mount
[ ] 37% Configure X11
[ ] 50% Mount data & params
[ ] 62% Add hardware devices

Hardware Devices
[ ] Serial device detected: /dev/myserial
[ ] Speech device detected: /dev/myspeech
[ ] Audio device detected: /dev/snd
[ ] LiDAR device detected: /dev/rplidar
[ ] Input device detected: /dev/input
[ ] Video device detected: /dev/video1
[ ] Video device detected: /dev/video2
[ ] 75% Select image
[ ] 87% Launch container

Launch Info
Image:      ros2-humble-muto:3.1.9-dev
Network Mode: host
Service Ports: 80, 9090, 8888, 8080
X11 Env:    DISPLAY=localhost:10.0

Container started successfully! Container ID: 2027f2d51643
[ ] 100% Launch success

Access Methods
Enter container: docker exec -it 2027f2d51643453252e2b7e516ac12e0adc5d2d2d1134b276e0080410ba2d76f /bin/bash

Container ready!
-----
ROS_DOMAIN_ID: 38
my_robot_type: Muto | my_lidar: a1
-----
root@raspberrypi:~#

```

In this tutorial, we use the voice version of the large model. **Do not** include any parameters when starting it.

```
./rebuild_and_launch.sh
```

```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

# Getting Started

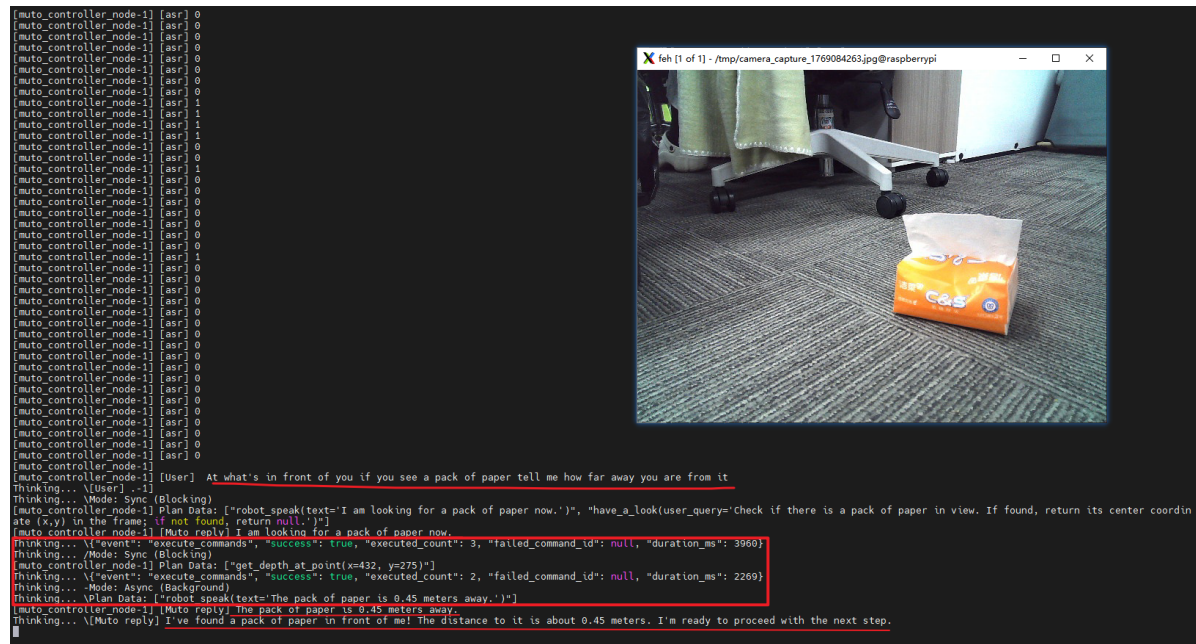
In this tutorial, we will integrate the large model with our depth camera.

## Usage Example

The command used in this section is: "Look at what's in front of you. If you see a pack of paper, tell me how far away you are from it."

Input this command via voice.

You can see a visualized image has opened.



Here we have measured the actual distance.

Note! When using the Astra Pro plus camera on MutoRS for distance measurement, the object needs to be more than 0.8cm away. Otherwise, the obtained depth data may be considered invalid and discarded.

You can change the object in your query based on the actual scene.

## Code Briefing

The key function used in this section is:

`get_depth_at_point(x, y)`: Get the distance at the specified coordinates (x, y) in the depth camera view (unit: meters). This needs to be used in conjunction with visual recognition results.

The function is defined in the path

`\packages\muto_hexapod_lib\muto_hexapod_lib\Largemodel\sensor_manager.py`.

The core code is as follows:

```
def get_depth_at_point(self, x: int, y: int, timeout_s: float = 5.0) -> tuple:
    """
    Get depth information at specified coordinates (with intelligent node management)
```

```

Args:
    x: X coordinate in the image
    y: Y coordinate in the image
    timeout_s: Timeout in seconds

Returns:
    tuple: (success: bool, message: str, data: dict)
        success: Whether execution was successful
        message: Execution result message
        data: Depth information
"""
depth_topic = "/camera/depth/image_raw"

try:
    # Intelligent node management: check initial camera status
    camera_was_running = False
    camera_started_by_us = False

    camera_success, camera_msg, camera_data = self.get_camera_status()
    camera_was_running = camera_success and camera_data.get('status') ==
'running'

    # If camera is not running, start it first
    if not camera_was_running:
        start_success, start_msg, start_data =
self.start_camera(wait_time_s=5.0)
        if not start_success:
            return False, f"无法启动相机 Failed to start camera:
{start_msg}", {
                "x": x,
                "y": y,
                "camera_start_attempted": True,
                "camera_start_success": False,
                "error": "camera_start_failed"
            }
        camera_started_by_us = True

    # Execute core depth acquisition function
    # Starting to get depth info at point
    # Depth topic

    # Check for necessary dependencies
    try:
        import rclpy
        from rclpy.node import Node
        from sensor_msgs.msg import Image
        from cv_bridge import CvBridge
        import cv2
        import numpy as np
        # All required dependencies available
    except ImportError as import_error:
        # Missing required dependencies
        # Please install: pip install opencv-python
        return False, f"缺少必要依赖 Missing required dependencies:
{str(import_error)}", {"x": x, "y": y, "error": "missing_dependencies"}

    # Create depth image capture node
    import uuid

```

```

depth_node_id = str(uuid.uuid4())[:8] # Use first 8 chars of UUID
as unique ID

class DepthCaptureNode(Node):
    def __init__(self):
        super().__init__(f'depth_capture_node_{depth_node_id}')
        self.bridge = CvBridge()
        self.depth_received = False
        self.depth_image = None
        self.subscription = self.create_subscription(
            Image,
            depth_topic,
            self.depth_callback,
            10
        )
        # Subscribed to depth topic

    def depth_callback(self, msg):
        try:
            # Convert ROS depth image message to OpenCV image
            # Depth image is usually 16-bit single channel
            depth_image = self.bridge.imgmsg_to_cv2(msg,
desired_encoding="passthrough")
            self.depth_image = depth_image
            self.depth_received = True
            # Successfully received depth image
        except Exception as e:
            print(f"深度图像转换失败 Depth image conversion failed:
{str(e)}")

            # Depth image conversion failed

# Initialize ROS2 (if not already initialized)
if not rclpy.ok():
    rclpy.init()

# Create node and wait for depth image
node = DepthCaptureNode()

# waiting for depth image data

# wait for depth image data - add error handling and retry mechanism
start_time = time.time()
spin_retry_count = 0
max_spin_retries = 3

while not node.depth_received and (time.time() - start_time) <
timeout_s:
    try:
        rclpy.spin_once(node, timeout_sec=0.1)
        spin_retry_count = 0 # Reset retry count
    except Exception as spin_error:
        spin_retry_count += 1
        error_msg = str(spin_error)

        # Check for "event index too big" error
        if "event index too big" in error_msg or "index" in
error_msg.lower():
            if spin_retry_count <= max_spin_retries:

```

```

        print(f"⚠️ ROS2事件循环错误 (深度获取重试
{spin_retry_count}/{max_spin_retries}): {error_msg}")
        time.sleep(0.2) # 短暂延迟后重试
        continue
    else:
        print(f"❌ ROS2事件循环错误超过最大重试次数 (深度获取):
{error_msg}")

        node.destroy_node()
        return False, f"ROS2事件循环错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_event_loop_error_depth"}
    else:
        # Other types of errors
        print(f"❌ ROS2 spin_once错误 (深度获取): {error_msg}")
        node.destroy_node()
        return False, f"ROS2 spin_once错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_spin_error_depth"}

if node.depth_received and node.depth_image is not None:
    # Check if coordinates are within image bounds
    height, width = node.depth_image.shape[:2]

    if x < 0 or x >= width or y < 0 or y >= height:
        # Coordinates out of image bounds
        # Image size
        # Requested coordinates
        node.destroy_node()
        return False, f"坐标超出图像范围 Coordinates out of bounds:
({x}, {y}), image size: {width}x{height}", {
            "x": x, "y": y,
            "image_width": width,
            "image_height": height,
            "error": "coordinates_out_of_bounds"
        }

    # Get depth value at specified coordinates
    depth_value = node.depth_image[y, x]

    # Depth value is usually in millimeters, convert to meters
    if node.depth_image.dtype == np.uint16:
        # 16-bit depth image, usually in mm
        depth_meters = float(depth_value) / 1000.0
        depth_mm = float(depth_value)
    elif node.depth_image.dtype == np.float32:
        # 32-bit float depth image, usually in meters
        depth_meters = float(depth_value)
        depth_mm = float(depth_value) * 1000.0
    else:
        # Other formats, assume mm
        depth_meters = float(depth_value) / 1000.0
        depth_mm = float(depth_value)

    # Check if depth value is valid (0 usually means invalid)
    is_valid = bool(depth_value > 0) # Convert to native Python
    bool

    # Depth information retrieved successfully
    # Coordinates
    # Raw depth value

```

```

# Depth (meters)
# Depth (millimeters)
# Data type
# Valid

# Get depth info for surrounding area (3x3 region)
surrounding_depths = []
for dy in range(-1, 2):
    for dx in range(-1, 2):
        nx, ny = x + dx, y + dy
        if 0 <= nx < width and 0 <= ny < height:
            surrounding_depth = node.depth_image[ny, nx]
            if node.depth_image.dtype == np.uint16:
                surrounding_depth_m = float(surrounding_depth) /
1000.0

            elif node.depth_image.dtype == np.float32:
                surrounding_depth_m = float(surrounding_depth)
1000.0
            else:
                surrounding_depth_m = float(surrounding_depth) /

            surrounding_depths.append({
                "x": nx, "y": ny,
                "depth_raw": float(surrounding_depth),
                "depth_meters": surrounding_depth_m,
                "valid": bool(surrounding_depth > 0) # Convert
to native Python bool
            })

# Clean up node
node.destroy_node()

# Intelligent node management: if we started the camera, close it
after execution
if camera_started_by_us:
    try:
        self.stop_camera(wait_time_s=1.0)
    except Exception as stop_error:
        print(f"⚠️ 自动关闭相机警告: {stop_error}")

    return True, "深度信息获取成功 Depth information retrieved
successfully", {
        "x": x,
        "y": y,
        "depth_raw": float(depth_value),
        "depth_meters": depth_meters,
        "depth_millimeters": depth_mm,
        "is_valid": is_valid,
        "image_size": {"width": width, "height": height},
        "data_type": str(node.depth_image.dtype),
        "surrounding_area": surrounding_depths,
        "method": "cv_bridge_depth",
        "camera_auto_managed": camera_started_by_us
    }
else:
    # Depth image capture timeout
    # Please check if depth camera is publishing data normally
    node.destroy_node()

```



```

        # Intelligent node management: if we started the camera, close it
        after execution
        if camera_started_by_us:
            try:
                self.stop_camera(wait_time_s=1.0)
            except Exception as stop_error:
                print(f"⚠ 自动关闭相机警告: {stop_error}")

        return False, f"深度图像获取超时 Depth image capture timeout
({timeout_s}s)", {
            "x": x, "y": y,
            "timeout": timeout_s,
            "suggestion": "check_depth_camera_publishing",
            "camera_auto_managed": camera_started_by_us
        }

    except Exception as e:
        # Error during depth information retrieval
        # Intelligent node management: if we started the camera, close it
        after execution
        if camera_started_by_us:
            try:
                self.stop_camera(wait_time_s=1.0)
            except Exception as stop_error:
                print(f"⚠ 自动关闭相机警告: {stop_error}")

        return False, f"深度信息获取过程中发生错误 Error during depth information
retrieval: {str(e)}", {
            "x": x, "y": y,
            "error": str(e),
            "camera_auto_managed": camera_started_by_us
        }

```